Documento de Requerimientos de Producto (PRD)

# Sistema de Análisis Estadístico

## para Básquet Formativo

Estado: **Borrador Técnico** | Versión: **1.0**

Ingeniería de Software · Proyecto Estadísticas Básquet

15 de junio de 2026

---

| | |
|---|---|
| **ID del Documento** | PRD-BSKT-2026-001 |
| **Producto** | StatsPro Basketball (*nombre en clave*) |
| **Ingeniería** | Equipo de 2 Ingenieros de Software |
| **Clasificación** | Interno — Desarrollo Profesional |
| **Stack Principal** | SQLite · Python/Pandas · Flet (UI) |

# Índice


**1. Descripción General del Producto** **4**
- 1.1. Planteamiento del Problema . . . 4
- 1.2. Visión del Producto . . . 4
- 1.3. Métricas de Éxito . . . 4

**2. Perfiles de Usuario** **5**

**3. Metas y No Metas** **5**
- 3.1. En el Alcance (v1.0) . . . 5
- 3.2. Fuera de Alcance . . . 5

**4. Acuerdo de Ingeniería y Estándares** **5**
- 4.1. Principios de Desarrollo . . . 5
- 4.2. Calidad de Código y Pruebas . . . 5

**5. Gestión de Tareas** **6**

**6. Arquitectura de Datos** **6**
- 6.1. Entidades Principales . . . 6
- 6.2. Reglas de Integridad . . . 6

**7. Arquitectura de Software (Clean + Hexagonal)** **6**
- 7.1. Capas y Responsabilidades . . . 6
- 7.2. Regla de Dependencias . . . 6
- 7.3. Patrones de Diseño . . . 7
- 7.4. Estructura de Directorios . . . 7
- 7.5. ¿Qué va en cada capa? Guía práctica . . . 7
  - 7.5.1. Capa de Dominio (`src/domain/`) . . . 7
  - 7.5.2. Capa de Aplicación (`src/application/`) . . . 8
  - 7.5.3. Capa de Infraestructura (`src/infrastructure/`) . . . 8
- 7.6. Data Transfer Objects (DTOs) . . . 8
- 7.7. Gestión de Sesión y Club Activo . . . 9
- 7.8. Composition Root: Cómo se Ensambla la Aplicación . . . 10

**8. Registro de Decisiones Arquitectónicas (ADR)** **10**

**9. Gestión de Tareas y Plan de Entregas** **11**
- 9.1. Hito 1 — Núcleo de Datos e Interfaz CLI . . . 11
- 9.2. Hito 2 — Motor de Ingesta y Análisis . . . 20
- 9.3. Hito 3 — Visualización Pro y Reporting . . . 25
- 9.4. Hito 4 — Interfaz Multiplataforma y Entrega . . . 28

**10. Reglas de Negocio Consolidadas** **32**

**11. Requisitos No Funcionales (NFR)** **32**

**12. Definición de "Hecho" (Definition of Done)** **33**

12.1. Catálogo Técnico de Criticidad . . . . . 33

**13. Proceso de Liberación de Versiones** **34**

13.1. Versionado Semántico . . . . . 34

13.2. Estrategia de Ramas . . . . . 34

13.3. Proceso de Release Paso a Paso . . . . . 34

13.4. Formato del `CHANGELOG.md` . . . . . 35

13.5. Hotfix: Corregir un Bug en Producción . . . . . 35

**14. Hoja de Ruta y Futuras Expansiones** **36**

# 1 Descripción General del Producto

## 1.1 Planteamiento del Problema

El análisis estadístico en el básquet formativo sufre una brecha crítica entre la recolección de datos y su utilidad táctica. Los entrenadores operan bajo alta presión donde la toma de decisiones basada en datos se ve limitada por herramientas fragmentadas.

Tabla 1: Matriz de Problemas Operativos

| # | Problema | Impacto Operativo | Línea Base |
|---|---|---|---|
| 1 | Carga manual ineficiente. | Interferencia táctica; pérdida de foco durante el partido. | >20 min/partido |
| 2 | Ceguera estadística. | Decisiones por intuición, sin métricas avanzadas (PPP, Eff). | 0 % automatizado |
| 3 | Fragmentación de datos. | Imposibilidad de seguimiento histórico o comparativo de rivales. | Papel / Excel |
| 4 | Inestabilidad de red. | Apps web fallan en estadios sin conectividad. | 100 % online |
| 5 | Rigidez de plataformas. | El DT no puede integrar datos de Ges Deportivo fácilmente. | Ingesta manual |

## 1.2 Visión del Producto

Desarrollar una aplicación multiplataforma (PC y Mobile) de ejecución local que centralice el conocimiento deportivo en un motor estadístico profesional. El sistema permitirá una gestión integral **sin necesidad de internet**, con un flujo optimizado mediante la carga de planillas de "Ges Deportivo".

## 1.3 Métricas de Éxito

- **MTTR (Time to Report):** tiempo desde la carga del archivo hasta el informe completo <2 minutos.
- **Tasa de Automatización:** % de estadísticas de partido generadas sin intervención manual >95 %.
- **Disponibilidad Offline:** 100 % de las funcionalidades críticas operativas sin conexión.
- **North Star Metric:** número de sesiones de análisis táctico realizadas por el DT por semana.

# 2 Perfiles de Usuario

| Perfil | Rol y Contexto | Objetivo Principal | Problema Actual |
|---|---|---|---|
| **Director Técnico** | Líder táctico en categorías formativas a profesional. | Optimizar rendimiento mediante datos objetivos. | El cálculo manual de Eff/PPP es tedioso. |
| **Analista / Asistente** | Responsable de carga y procesamiento de datos. | Proveer informes rápidos y precisos al DT. | Carga redundante de datos ya existentes en Ges Deportivo. |

# 3 Metas y No Metas

## 3.1 En el Alcance (v1.0)

- **Ingesta Multimodal:**
  - *Automática:* carga de planillas Excel de "Ges Deportivo".
  - *Manual:* formulario para carga post-partido (jugador por jugador).
- **Persistencia Local:** base de datos SQLite única por instalación de usuario.
- **Motor Estadístico:** análisis de tendencias, PPP, porcentajes por zona y eficiencia con Pandas.
- **Multiplataforma:** ejecución en Windows, Linux, macOS y dispositivos móviles.

## 3.2 Fuera de Alcance

- **Carga en Vivo (v1.0):** la funcionalidad de toma de datos en tiempo real se posterga a versiones post-estables.
- **Sincronización Cloud:** no se contempla almacenamiento en la nube inicialmente.
- **Importación Automática/API:** no hay conexión directa con servidores de Ges Deportivo.

# 4 Acuerdo de Ingeniería y Estándares

## 4.1 Principios de Desarrollo

- **Diseño Precede a la Implementación:** no se escribe código sin un diseño previo aprobado (ADR).
- **Atomicidad:** commits pequeños y lógicos. Formato: `tipo(alcance): descripción`.
- **Gestión de Ramas:** `feature/nombre-tarea`, `hotfix/descripción`, `release/vX.Y`.
- **Higiene del Repositorio:** prohibido subir binarios, bases SQLite con datos reales o archivos temporales.

## 4.2 Calidad de Código y Pruebas

- **Documentación:** estilo Javadoc/Doxygen para módulos y métodos públicos.
- **Pruebas:** cobertura mínima del 80 % en lógica de negocio; 95 % en componentes críticos.
- **Análisis Estático:** `flake8`/`pylint` para Python; `Checkstyle` para Java.

# 5 Gestión de Tareas

| Prioridad | Descripción |
|---|---|
| **Urgente** | Bloqueante; detiene el desarrollo de otras US. |
| **Alta** | Impacto directo en la entrega del hito. |
| **Media** | Importante pero no bloquea el progreso. |
| **Baja** | Mejora o refinamiento posterior. |

| Tamaño | Esfuerzo estimado |
|---|---|
| **XS** | Menos de 1 día hábil. |
| **S** | 1–2 días hábiles. |
| **M** | 3–5 días hábiles. |
| **L** | 6–10 días hábiles. |

# 6 Arquitectura de Datos

El sistema utiliza una base de datos relacional local (**SQLite**) con esquema normalizado.

## 6.1 Entidades Principales

- **Gestión de Identidad:** `usuario`, `club`, `usuarioClub` (N:M).
- **Estructura Deportiva:** `jugador`, `categoria`, `competencia`, `inscripcion`.
- **Gestión de Listas:** `listaBuenaFe`, `jugadorListaBuenaFe`.
- **Eventos y Estadísticas:** `partido`, `jugadorPartido` (20+ métricas: Puntos, T2, T3, TL, Rebotes, Asistencias, etc.).

## 6.2 Reglas de Integridad

- **Vínculos:** relación 1:1 entre `inscripcion` y `listaBuenaFe`.
- **Persistencia:** historial de afiliaciones de un jugador se mantiene vía `jugadorClub`.
- **Métricas:** la tabla `jugadorPartido` actúa como *fact table* para el motor de análisis.

# 7 Arquitectura de Software (Clean + Hexagonal)

## 7.1 Capas y Responsabilidades

- **Dominio** (`src/domain`): entidades puras, reglas de negocio, excepciones e interfaces (ports), sin dependencias externas.
- **Aplicación** (`src/application`): casos de uso y DTOs; orquesta reglas mediante inyección de dependencias.
- **Infraestructura** (`src/infrastructure`): repositorios SQLite, parser Excel (Pandas), UI CLI/GUI y generación de reportes.

## 7.2 Regla de Dependencias

`domain` ← `application` ← `infrastructure`

- `domain` no puede importar `application` ni `infrastructure`.
- `application` puede importar `domain`, pero no `infrastructure`.

## 7.3 Patrones de Diseño

- **Repository Pattern:** abstracción de persistencia por agregado (`UserRepository`, `PlayerRepository`, etc.).
- **Dependency Injection:** los casos de uso reciben puertos, no implementaciones concretas.
- **Command Pattern:** CLI extensible por subcomandos sin bloques monolíticos `if/else`.

## 7.4 Estructura de Directorios

- `src/domain/entities/` · `repositories/` · `exceptions/`
- `src/application/use_cases/` · `dtos/` · `services/`
- `src/infrastructure/repositories/` · `persistence/` · `analytics/` · `ingest/` · `reports/` · `ui/` · `security/` · `logging/`
- `tests/unit/` · `tests/integration/` · `tests/fixtures/`
- `docs/arquitectura/` · `docs/guias/` · `docs/security/`

**Convención de repositorios**

Las **interfaces** (contratos abstractos) de repositorios viven en `src/domain/repositories/`. Las **implementaciones concretas** en SQLite viven en `src/infrastructure/repositories/`. Esta separación es la que hace posible testear los casos de uso sin base de datos real.

## 7.5 ¿Qué va en cada capa? Guía práctica

### 7.5.1 Capa de Dominio (`src/domain/`)

Es el **núcleo del sistema**. No depende de ninguna tecnología concreta. Si se cambia SQLite por PostgreSQL, o la CLI por una GUI, esta capa *no se toca*.

- **`entities/`** — Clases Python puras (`@dataclass` o clases normales) que modelan los conceptos del básquet. Pueden contener métodos con lógica de negocio simple (validaciones, cálculos derivados). **No importan** `sqlite3`, `pandas` ni ningún framework.

  Ejemplo: `Jugador` tiene campos `nombre`, `dni`, `fecha_nacimiento` y un método `calcular_edad()`. `EstadisticaJugador` tiene los 20+ rubros de boxscore y un método `validar_consistencia_puntos()`.

- **`repositories/`** — Interfaces abstractas (clases que heredan de `ABC` con `@abstractmethod`) que definen *qué* operaciones existen sobre los datos, sin decir *cómo* se hacen.

  Ejemplo: `PlayerRepository` declara métodos como `find_by_id(id)`, `find_by_dni(dni)`, `save(jugador)`, `find_by_club(club_id)`. No hay una sola línea de SQL. La capa de aplicación solo conoce estos contratos, nunca la implementación.

- **`exceptions/`** — Errores propios del negocio deportivo. Permiten que los casos de uso lancen errores descriptivos sin acoplar la UI a detalles técnicos.

  Ejemplo: `DNIDuplicadoError`, `JugadorNoHabilitadoError`, `PartidoInvalidoError("El club local no puede ser el visitante")`.

### 7.5.2 Capa de Aplicación (src/application/)

Orquesta el dominio para satisfacer los casos de uso del usuario. No contiene lógica de negocio pura (eso va en dominio) ni detalles técnicos (eso va en infraestructura).

- `use_cases/` — **Un archivo = una acción del usuario**. Cada caso de uso recibe sus dependencias (repositorios, servicios) como argumentos en el constructor (inyección de dependencias), y expone un único método `execute(dto)`. El caso de uso coordina: valida el DTO entrante, invoca entidades de dominio, llama al repositorio y retorna un DTO de salida.

  Ejemplo: `RegistrarJugadorUseCase.execute(dto)` verifica que el DNI no exista (`player_repo.find_by_dni`), crea la entidad `Jugador`, la persiste (`player_repo.save`) y retorna un `JugadorDTO`.

- `dtos/` — Ver Sección 7.6.

- `services/` — Servicios transversales de aplicación que no pertenecen a un caso de uso específico, como `SessionManager` (gestión de sesión persistente) y `ExecutionContext` (propagación de `correlation_id` para logs).

### 7.5.3 Capa de Infraestructura (src/infrastructure/)

Implementa los contratos del dominio usando tecnologías concretas. Es la única capa que puede importar `sqlite3`, `pandas`, `flet`, `bcrypt`, etc.

- `repositories/` — Implementaciones SQLite de las interfaces de dominio. Cada clase hereda de su interfaz correspondiente (`SQLitePlayerRepository(PlayerRepository)`) e implementa todos los métodos abstractos con SQL real.

- `persistence/` — Infraestructura técnica de la base de datos: `database_manager.py` (conexión, inicialización), archivos `.sql` (schema, vistas, seeds) y migraciones. No contiene lógica de negocio.

- `analytics/` — Servicios Pandas: `formulas.py` (funciones puras sobre DataFrames), `pandas_analytics_service` (conecta vistas SQL con las fórmulas), `chart_generator.py`.

- `ingest/` — Parser Excel de Ges Deportivo y servicio de ingesta.

- `reports/` — Generadores de PDF y reportes de CLI (leaderboards).

- `ui/` — Subárbol separado por interfaz: `ui/cli/` para la CLI y `ui/flet/` para la GUI. Llaman a casos de uso; nunca acceden a la base de datos directamente.

- `security/` — Hashing de contraseñas, política de credenciales, adaptador de cifrado de DB.

- `logging/` — Configuración de sinks de log (archivo local / Seq).

## 7.6 Data Transfer Objects (DTOs)

Un **DTO** (Data Transfer Object) es una clase simple cuya única responsabilidad es transportar datos entre capas. **No contiene lógica de negocio**.

**¿Por qué los usamos?**

- **Desacoplamiento:** la UI no necesita conocer las entidades de dominio, y el dominio no se expone directamente al exterior.
- **Control de la frontera:** el DTO de entrada valida el formato de los datos antes de que el caso de uso los procese. El DTO de salida define exactamente qué información se devuelve.
- **Evolución independiente:** se puede cambiar la entidad de dominio sin romper la UI, y viceversa.

**Patrón de uso en este proyecto:** cada caso de uso recibe un **DTO de entrada** (datos crudos del usuario) y retorna un **DTO de salida** (datos procesados para mostrar). Las entidades de dominio *nunca salen* de la capa de aplicación hacia la UI.

**Ejemplo concreto** — Registrar un jugador:

- `RegistrarJugadorInputDTO`: contiene `nombre`, `apellido`, `dni`, `fecha_nacimiento`, `club_id`. Solo datos planos, sin métodos.
- El caso de uso recibe este DTO, crea la entidad `Jugador` con las reglas de dominio, y persiste.
- Retorna `JugadorOutputDTO`: contiene `id`, `nombre_completo`, `dni`, `club_nombre`. Solo lo que la CLI/GUI necesita mostrar.

**Ubicación en el proyecto:** `src/application/dtos/`. Convención de nombres: `jugador_dto.py` puede contener tanto el DTO de entrada como el de salida para esa entidad.

### 7.7 Gestión de Sesión y Club Activo

La sesión local persiste en un archivo JSON en `~/.statspro/session.json` (o `./data/session.json` en desarrollo). El `SessionManager` es el único responsable de leer y escribir este archivo.

**Estructura del archivo de sesión:**

- `usuario_id`: identificador del usuario autenticado.
- `email`: email del usuario (para mostrar en la UI).
- `club_activo_id`: ID del club seleccionado actualmente. Puede ser `null` si no se seleccionó ninguno.
- `session_token_hash`: hash del token de sesión (nunca el token en texto plano).
- `expires_at`: timestamp ISO 8601 de expiración de la sesión.

**¿Cómo se establece el `club_activo_id`?**

1. Al hacer login, el `SessionManager` crea la sesión con `club_activo_id = null`.
2. El usuario ejecuta `stats club select <id>`.
3. El comando llama al caso de uso `CambiarClubActivoUseCase`, que verifica que el club pertenezca al usuario.
4. Si la verificación pasa, el `SessionManager` actualiza `club_activo_id` en el archivo de sesión.
5. Los comandos operativos (cargar partido, listar jugadores, etc.) leen `club_activo_id` de la sesión al inicio; si es `null`, abortan con un mensaje claro.

**¿Por qué club activo y no pasarlo como argumento?**

Un DT trabaja siempre en el contexto de un club. Tener el club activo en la sesión evita que el usuario tenga que escribir `-club-id 3` en cada comando. Es el mismo patrón que usan IDEs con el "proyecto activo" o shells con el "directorio actual".

### 7.8 Composition Root: Cómo se Ensambla la Aplicación

El **Composition Root** es el único lugar del sistema donde se instancian todas las dependencias y se conectan entre sí. En nuestra arquitectura, ese lugar es `src/infrastructure/ui/cli/main_cli.py` (CLI) y `src/infrastructure/ui/flet/app.py` (GUI).

**¿Por qué es importante?** Porque en todos los demás archivos, las clases reciben sus dependencias como argumentos (nunca las crean con `new`/instanciación directa). Esto hace el sistema testeable: en los tests se pueden pasar repositorios falsos (*mocks*) sin modificar el código de producción.

**Flujo de ensamblaje en `main_cli.py`** (crece con cada US):

1. **US-101/102**: Se instancia `SQLiteManager` y se ejecutan las migraciones. Se crean los repositorios SQLite pasando la conexión.
2. **US-103/104**: Se crean los servicios de aplicación (`SessionManager`) y los casos de uso administrativos, pasando los repositorios.
3. **US-105/106**: Se crean los casos de uso operativos y se registran todos los subcomandos CLI, pasando los casos de uso y el `SessionManager`.
4. **US-107**: Se inicializa el `ExecutionContext` antes de despachar cualquier comando y se conecta el sistema de logging.
5. **US-401 (GUI)**: En `app.py`, mismo patrón pero las pantallas Flet reciben los casos de uso como dependencias.

**Regla de oro:** si una clase crea sus dependencias con `NombreClase()` dentro de un método que no sea el composition root, hay un problema de acoplamiento que debe corregirse.

## 8 Registro de Decisiones Arquitectónicas (ADR)

| ID | Título | Estado | Decisión | Bloquea |
|---|---|---|---|---|
| ADR-001 | Arquitectura Local-First | **Aprobado** | SQLite + offline-first. | Hito 1 |
| ADR-002 | Framework UI | **Pendiente** | Selección entre Flet y Compose Multiplatform. | Hito 4 |
| ADR-003 | Protocolo de Ingesta | **Pendiente** | Estandarizar mapeo y limpieza de Excel Ges Deportivo. | US-201 |
| ADR-004 | Versionado de DB | **Pendiente** | Definir migraciones (`schema_version` o herramienta dedicada). | Hito 2 |
| ADR-005 | Reportes PDF | **Pendiente** | Seleccionar librería PDF local (ej. ReportLab). | US-302 |
| ADR-006 | Seguridad y Cifrado | **Pendiente** | Hash de contraseñas y eventual cifrado DB (SQLCipher). | Hito 4 |
| ADR-007 | Motor de Visualización | **Pendiente** | Selección de librería de gráficos para dashboards. | US-301 |
| ADR-008 | Estrategia de Backup | **Pendiente** | Exportación/restauración de base local y versiones. | Hito 4 |

| ID | Título | Estado | Decisión | Bloquea |
|---|---|---|---|---|
| ADR-009 | Pipeline CI/CD | **Pendiente** | GitHub Actions para lint, tests y cobertura automáticos. | US-108 |

# 9 Gestión de Tareas y Plan de Entregas

**Nota Operativa**

Los criterios de aceptación en este PRD son plantillas para crear issues en el tablero de trabajo. El estado real (`[ ]`/`[x]`) se gestiona en el board de desarrollo, no en este archivo versionado.

## 9.1 Hito 1 — Núcleo de Datos e Interfaz CLI

**H1 · Núcleo de Datos e Interfaz CLI** v0.1

Construir una base técnica funcional por CLI con persistencia robusta, autenticación local, casos de uso operativos, validaciones de integridad y un entorno de calidad que garantice reproducibilidad desde el primer commit.

**Épicas:** 3 **Historias:** 8 **Esfuerzo total:** ≈50 días · persona

### H1-E1 · Infraestructura y Persistencia

#### US-101 — Esquema SQLite, Vistas y Datos Semilla

**Esfuerzo:** L (6–10 días) **Prioridad:** Urgente BLOQUEANTE **Dependencias:** —

**Objetivo Funcional**

Habilitar un esquema relacional local verificable y un conjunto de vistas operativas que sirvan de contrato estable para todo el ciclo del producto, garantizando que Pandas y la capa de aplicación consuman datos sin transformaciones ambiguas.

**Archivos a Crear**

- `src/infrastructure/persistence/database_manager.py` — Clase `SQLiteManager`. Responsable de abrir la conexión a SQLite, activar FKs con `PRAGMA foreign_keys = ON`, establecer `row_factory` y ejecutar el schema inicial. Es el único lugar que conoce la ruta del archivo `.db`.
- `src/infrastructure/persistence/sql/schema.sql` — DDL completo del sistema. Define todas las tablas con `CREATE TABLE IF NOT EXISTS`, claves foráneas, `CHECK` constraints y comentarios explicativos por tabla.
- `src/infrastructure/persistence/sql/views.sql` — Las 4 vistas operativas que actúan como API SQL del sistema. Toda la lógica de consulta compleja vive aquí, no en el código

Python.

- `src/infrastructure/persistence/sql/seed.sql` — Datos de referencia para desarrollo y testing. Permite arrancar el sistema con datos realistas sin cargar archivos Excel manualmente.
- `tests/integration/test_database_schema.py` — Smoke-tests que verifican que el schema se aplica sin errores, todas las tablas y vistas existen, y los datos semilla tienen la cardinalidad esperada.
- `docs/arquitectura/00-base-de-datos.md` — Documentación del schema: diagrama de entidades (texto), explicación de cada tabla y sus relaciones, y guía de las 4 vistas con ejemplos de consulta.

### Vistas a Implementar

- `v_partidos_resumen`: una fila por partido con competencia, fecha, clubes, marcador y estado.
- `v_boxscore_completo`: una fila por jugador-partido con todos los rubros estadísticos crudos.
- `v_jugador_totales_temporada`: acumulados y promedios por jugador-temporada-competencia.
- `v_listas_detalle`: jugadores habilitados por inscripción/lista de buena fe con estado y club.

### Criterios de Aceptación

**AC1.** Schema idempotente con `CREATE TABLE IF NOT EXISTS` en toda la DDL.
**AC2.** FKs activas con reglas `ON DELETE/UPDATE CASCADE` definidas en relaciones críticas.
**AC3.** `CHECK constraints` aplicadas en métricas numéricas y campos de integridad lógica.
**AC4.** Las 4 vistas operativas exponen columnas con nombres y tipos estables, documentados.
**AC5.** Todas las divisiones en vistas están protegidas con `NULLIF`/`CASE` (nunca error por división por cero).
**AC6.** Datos semilla: 1 usuario, 2 clubes, 10 jugadores (5 por club), 1 competencia, 3 partidos con boxscore completo.
**AC7.** Columnas de vistas estables para consumo desde Pandas sin transformaciones adicionales.

### Reglas de Negocio

- `idClubLocal != idClubVisitante` en toda inserción de partido.
- Coherencia temporal: `fechaHasta >= fechaDesde` en todas las afiliaciones.
- Unicidad en `listaBuenaFe.idInscripcion`.

### Testing Mínimo

- *Integración — schema:* ejecución limpia y sin errores de `schema.sql`, `views.sql`, `seed.sql`.
- *Integración — tablas/vistas:* existencia verificada de todas las tablas, vistas, FKs y constraints.
- *Integración — semántica por vista:* assert de columnas esperadas y cardinalidad con datos semilla.
- *Integración — robustez:* división por cero en vistas devuelve `0.0` o `NULL` controlado, nunca excepción.

## US-102 — DatabaseManager y Patrón Repository

**Esfuerzo:** M (3–5 días) **Prioridad:** Urgente BLOQUEANTE **Dependencias:** US-101

### Objetivo Funcional

Separar el acceso a datos por agregados de dominio bajo contratos tipados, garantizando que la capa de dominio no tenga dependencias de `sqlite3` ni de `pandas`.

### Archivos a Crear

- `src/domain/repositories/user_repository.py` — Interfaz abstracta (hereda de `ABC`). Define los métodos del repositorio de usuarios con `@abstractmethod`: `find_by_email`, `find_by_id`, `save`. Sin ninguna mención a SQLite.
- `src/domain/repositories/club_repository.py` — Contrato para clubes y la relación `usuarioClub`: `find_by_id`, `find_by_usuario`, `save`, `link_user_to_club`.
- `src/domain/repositories/player_repository.py` — Contrato para jugadores y el historial `jugadorClub`: `find_by_id`, `find_by_dni`, `find_by_club`, `save`, `link_to_club`, `get_active_club`.
- `src/domain/repositories/competition_repository.py` — Contrato para el agregado competitivo (`competencia`, `categoria`, `inscripcion`, `listaBuenaFe`, `jugadorListaBuenaFe`): `save_competencia`, `find_competencia_by_id`, `get_all_competencias`, `save_categoria`, `get_categorias`, `save_inscripcion`, `find_inscripcion_by_id`, `get_inscripciones_by_club`, `save_lista_buena_fe`, `get_lista_by_inscripcion`, `add_jugador_to_lista`, `get_jugadores_by_lista`.
- `src/domain/repositories/game_repository.py` — Contrato para partidos y boxscore (`partido`, `jugadorPartido`): `save_with_boxscore` (transaccional), `find_by_club`, `find_by_id`.
- `src/infrastructure/repositories/sqlite_user_repository.py` — Implementación concreta que hereda de `UserRepository`. Aquí vive el SQL real. Recibe un `SQLiteManager` en el constructor.
- `src/infrastructure/repositories/sqlite_club_repository.py` — Ídem para clubes. Maneja `club` y la tabla de unión `usuarioClub`.
- `src/infrastructure/repositories/sqlite_player_repository.py` — Ídem para jugadores. Maneja `jugador` y el historial `jugadorClub`.
- `src/infrastructure/repositories/sqlite_competition_repository.py` — Maneja el agregado competitivo completo: `competencia`, `categoria`, `inscripcion`, `listaBuenaFe` y `jugadorListaBuenaFe`.
- `src/infrastructure/repositories/sqlite_game_repository.py` — Ídem para partidos. Maneja `partido` y `jugadorPartido`. Implementa la transacción atómica de `save_with_boxscore`.
- `tests/integration/test_repositories.py` — Tests CRUD contra SQLite `:memory:` para cada repositorio.
- `docs/arquitectura/01-repository-pattern.md` — Guía de estudio del patrón: qué es, por qué se usa, cómo agregar un repositorio nuevo al proyecto paso a paso.

### Criterios de Aceptación

**AC1.** `connect()` activa FKs y establece `row_factory=sqlite3.Row` antes de devolver la conexión.

**AC2.** Los repositorios retornan `dataclasses` tipados, no tuplas SQL crudas.

**AC3.** La capa `domain` no contiene ningún `import sqlite3` ni `import pandas`.

**AC4.** La operación guardar partido + boxscore completo se ejecuta en una única transacción atómica con rollback verificable.

### Testing Mínimo

- *Integración:* CRUD completo (crear, leer, actualizar, eliminar) por cada repositorio.
- *Integración:* verificación de mapeo correcto de columnas SQL a campos de dataclasses.
- *Integración:* atomicidad: inserción parcial forzada → rollback → estado DB sin cambios.

## H1-E2 · Lógica de Aplicación y CLI

### US-103 — Gestión de Entidades (Casos de Uso Administrativos)

**Esfuerzo:** L (6–10 días) **Prioridad:** Alta **Dependencias:** US-101, US-102

---

### Objetivo Funcional

Implementar la lógica de negocio para la gestión de jugadores, clubes, competencias e inscripciones, exponiéndola mediante casos de uso desacoplados de la persistencia.

### Archivos a Crear

*Entidades de dominio* — clases Python puras, sin imports de infraestructura:

- `src/domain/entities/usuario.py` — Entidad usuario con email, contraseña hasheada y rol. Método `validar_email()`.
- `src/domain/entities/club.py` — Entidad club con nombre y asociación a usuarios. Validación de nombre no vacío.
- `src/domain/entities/jugador.py` — Entidad jugador con DNI, nombre, apellido y fecha de nacimiento. Método `calcular_edad()` y validación de DNI.
- `src/domain/entities/jugador_club.py` — Vínculo temporal entre jugador y club (fecha desde/hasta). Valida coherencia de fechas.
- `src/domain/entities/competencia.py` — Competencia con nombre y temporada (año).
- `src/domain/entities/categoria.py` — Categoría deportiva (ej: U15, U17, Primera). Nombre y descripción.
- `src/domain/entities/inscripcion.py` — Inscripción de un club en una competencia+categoría. Unicidad por club+competencia+categoría.
- `src/domain/entities/lista_buena_fe.py` — Lista de buena fe asociada 1:1 a una inscripción.
- `src/domain/entities/jugador_lista_buena_fe.py` — Tabla de unión entre jugador y lista

de buena fe. Representa la habilitación de un jugador para una inscripción específica.

- `src/domain/entities/partido.py` — Partido con clubs local/visitante, fecha y marcador. Valida que los clubs sean distintos.
- `src/domain/entities/estadistica_jugador.py` — Los 20+ rubros del boxscore. Métodos de validación de consistencia (puntos, tiros convertidos/lanzados).

*Casos de uso* — un archivo por acción, reciben repositorios como argumentos:

- `src/application/use_cases/registrar_jugador.py` — Verifica DNI único, crea entidad `Jugador`, persiste via `PlayerRepository`, retorna `JugadorOutputDTO`.
- `src/application/use_cases/crear_club.py` — Crea club, lo asocia al usuario activo.
- `src/application/use_cases/vincular_jugador_club.py` — Crea `JugadorClub`, verifica que no haya vínculo activo superpuesto.
- `src/application/use_cases/crear_competencia.py`, `inscribir_club_competencia.py` — Gestión de competencias e inscripciones.
- `src/application/use_cases/listar_clubes_usuario.py` — Consulta a `ClubRepository`; retorna lista de `ClubDTO`.
- `src/application/use_cases/listar_jugadores_club.py`, `listar_partidos_por_club.py` — Ídem para jugadores y partidos.
- `src/application/use_cases/cambiar_club_activo.py` — Verifica que el club pertenezca al usuario, luego llama a `SessionManager.set_club_activo(id)`. Ver Sección 7.7.

*DTOs* — ver Sección 7.6 para el concepto. Un DTO por entidad, con variantes Input/Output:

- `src/application/dtos/jugador_dto.py` — `RegistrarJugadorInputDTO` (campos del formulario) y `JugadorOutputDTO` (datos para mostrar, incluye nombre del club).
- `src/application/dtos/club_dto.py`, `competencia_dto.py`, `inscripcion_dto.py`, `partido_resumen_dto.py`

*Tests y documentación:*

- `tests/unit/test_entities.py` — Pruebas de validaciones de dominio en cada entidad.
- `tests/unit/test_use_cases_admin.py` — Casos de uso con repositorios mockeados.
- `docs/arquitectura/02-casos-de-uso-y-dtos.md` — Guía explicando el patrón Use Case + DTO con ejemplos del proyecto: cómo crear un nuevo caso de uso, cómo definir sus DTOs, y cómo escribir el test.

### Criterios de Aceptación

**AC1.** Validación fail-fast de DNI inválido o duplicado antes de toda persistencia.

**AC2.** Control de afiliaciones activas: un jugador no puede tener dos vínculos activos superpuestos con el mismo club.

**AC3.** Alta de entidades con reglas de dominio completamente verificadas antes de persistir.

**AC4.** Operaciones de inscripción con comportamiento atómico (todas las tablas afectadas o ninguna).

**AC5.** Listados CLI tabulados y consistentes con los resultados de las vistas SQL.

### Testing Mínimo

- *Unitario:* validaciones de dominio en entidades (DNI, fechas, coherencia).

- *Unitario:* casos de uso con repositorios mockeados.
- *Integración:* casos de uso con SQLite `:memory:` verificando persistencia real.

### US-104 — Autenticación y Sesión Local

**Esfuerzo:** M (3–5 días) **Prioridad:** Alta **Dependencias:** US-103

#### Objetivo Funcional

Proveer registro y login seguros con persistencia de sesión entre ejecuciones, incluyendo el contexto de club activo para comandos operativos.

#### Archivos a Crear

- `src/infrastructure/security/password_hasher.py` — Wrapper sobre `bcrypt` (o `argon2-cffi`). Expone solo dos métodos: `hash(password)` y `verify(password, hash)`. El resto del sistema nunca llama a `bcrypt` directamente.
- `src/application/services/session_manager.py` — Lee y escribe `~/.statspro/session.json`. Campos del archivo: `usuario_id`, `email`, `club_activo_id` (puede ser `null`), `session_token_hash`, `expires_at`. Métodos clave: `create(usuario)`, `get_current()`, `set_club_activo(club_id)`, `destroy()`.
- `src/application/use_cases/registrar_entrenador.py` — Valida email único, aplica política de contraseña, hashea con `PasswordHasher`, persiste via `UserRepository`.
- `src/application/use_cases/login_local.py` — Busca usuario por email, verifica hash con `PasswordHasher.verify()`, crea sesión via `SessionManager`.
- `src/application/use_cases/cambiar_club_activo.py` — Verifica que el `club_id` exista y pertenezca al usuario de la sesión activa. Si pasa, llama a `session_manager.set_club_activo(club_id)`. Ver Sección 7.7 para el flujo completo.
- `src/application/dtos/auth_dto.py` — `RegisterInputDTO` (email, password), `LoginInputDTO` (email, password), `SessionDTO` (usuario_id, email, club_activo_id).
- `tests/unit/test_auth.py` — Tests de hash/verify, política de contraseña, registro duplicado, login inválido.
- `docs/arquitectura/03-sesion-y-autenticacion.md` — Guía del flujo completo: registro → login → selección de club → uso de comandos → logout. Incluye diagrama de secuencia en texto ASCII y explicación del archivo de sesión.

#### Comandos CLI

`stats auth register` · `stats auth login` · `stats auth logout` · `stats club select <id>`

#### Criterios de Aceptación

**AC1.** Las contraseñas nunca se persisten ni se loguean en texto plano.

**AC2.** La sesión se persiste en un archivo local con permisos restringidos (modo 600 en sistemas POSIX).

**AC3.** El contexto de sesión incluye `club_activo_id` y se actualiza al cambiar de club.

**AC4.** Validación de email único y política de contraseña mínima (longitud, complejidad) en el registro.

#### Testing Mínimo

- *Unitario:* hash/verificación de contraseñas; rechazo de contraseñas débiles.
- *Unitario:* registro duplicado (mismo email) retorna error de dominio.
- *Integración:* ciclo completo register → login → logout con archivo de sesión temporal.

### US-105 — Carga Atómica de Partido

**Esfuerzo:** M (3–5 días) **Prioridad:** Alta **Dependencias:** US-103, US-104

#### Objetivo Funcional

Persistir un partido y su boxscore completo en una operación todo-o-nada, garantizando que cualquier fallo parcial deje la base de datos en el estado anterior.

#### Archivos a Crear

- `src/application/dtos/partido_dto.py` — DTO con validación interna de métricas.
- `src/application/use_cases/cargar_partido.py` — orquestador del flujo de carga.
- `tests/unit/test_use_case_cargar_partido.py`
- `tests/integration/test_cargar_partido_atomicidad.py`

#### Criterios de Aceptación

**AC1.** Rollback completo ante cualquier error parcial del boxscore, sin registros huérfanos.
**AC2.** Validación completa de todos los DTOs antes de la primera operación de base de datos.
**AC3.** Mensajes de error identifican el jugador y el campo inválido de forma legible.
**AC4.** El caso de uso no contiene SQL embebido; delega toda persistencia al repositorio.

#### Reglas de Negocio

- Convertidos $\leq$ lanzados en todos los rubros de tiro.
- Minutos por jugador: no negativos y no superiores al máximo reglamentario.
- Puntos totales del jugador = T1C + T2C×2 + T3C×3.

#### Testing Mínimo

- *Unitario:* validación de DTO con cada regla de negocio violada (un test por regla).
- *Integración:* carga exitosa → verificar presencia en `v_boxscore_completo`.
- *Integración:* fallo en fila intermedia del boxscore → verificar rollback total.

### US-106 — CLI con Command Pattern

**Esfuerzo:** M (3–5 días) **Prioridad:** Alta **Dependencias:** US-103, US-104, US-105

### Objetivo Funcional

Construir una CLI extensible y mantenible con subcomandos desacoplados que consolide todo el flujo operativo de v0.1, sin bloques monolíticos `if/else`.

### Archivos a Crear

- `src/infrastructure/ui/cli/main_cli.py` — **Composition Root** de la aplicación CLI (ver Sección 7.8). Instancia el `SQLiteManager`, corre las migraciones, crea todos los repositorios, servicios y casos de uso, y registra todos los subcomandos. Es el único archivo que hace `NombreClase()` directamente. Cada nueva US que agregue un caso de uso debe actualizar este archivo para inyectarlo.
- `src/infrastructure/ui/cli/commands/auth_commands.py` — Subcomandos `stats auth *`. Recibe `RegistrarEntrenadorUseCase`, `LoginLocalUseCase` y `SessionManager` como dependencias. No instancia nada internamente.
- `src/infrastructure/ui/cli/commands/club_commands.py` — Subcomandos `stats club list`, `stats club select <id>`. Recibe `ListarClubesUseCase` y `CambiarClubActivoUseCase`.
- `src/infrastructure/ui/cli/commands/player_commands.py` — Subcomandos `stats player *`. Requiere `club_activo_id` en sesión.
- `src/infrastructure/ui/cli/commands/game_commands.py` — Subcomandos `stats game load`, `stats game list`. Requiere `club_activo_id` en sesión.
- `src/infrastructure/ui/cli/formatters/table_formatter.py` — Funciones utilitarias para imprimir tablas con columnas alineadas. Todos los comandos lo usan; centraliza el formato de salida.
- `docs/arquitectura/04-composition-root.md` — Guía del patrón DI y el composition root: por qué existe, qué se instancia dónde, y el proceso paso a paso para agregar un nuevo caso de uso al sistema (crear interfaz → implementar → agregar a `main_cli.py` → registrar comando).

### Criterios de Aceptación

**AC1.** Comandos protegidos verifican sesión autenticada antes de ejecutarse.
**AC2.** Comandos operativos verifican `club_activo_id` en la sesión cuando es necesario.
**AC3.** Listados de lectura consumen vistas SQL; no se generan consultas ad-hoc dispersas.
**AC4.** Errores funcionales muestran un mensaje legible sin traceback técnico al usuario.

### Testing Mínimo

- *Unitario:* flujo de guards de autenticación y club activo.
- *Integración:* ejecución de cada subcomando con DB en memoria; verificar salida esperada.
- *Integración:* comando sin sesión → mensaje de error; comando con sesión inválida → igual.

## US-107 — Monitoreo y Trazabilidad Operativa

**Esfuerzo:** M (3–5 días) **Prioridad:** Media **Dependencias:** US-106

### Objetivo Funcional

Proveer observabilidad transversal estructurada para depuración y soporte, con correlación de eventos extremo-a-extremo por ejecución de comando.

### Archivos a Crear

- `src/infrastructure/logging/logger_config.py` — configuración de sinks (local-file / Seq).
- `src/application/services/execution_context.py` — genera `correlation_id` UUIDv4 y propaga contexto.
- `src/infrastructure/logging/seq_handler.py` — handler HTTP para Seq (opcional).
- `docker-compose.yml` — perfil `observabilidad` con Seq (opcional).
- `tests/unit/test_execution_context.py`
- `tests/unit/test_logging_redaction.py`

### Eventos de Log Obligatorios

- Inicio/fin de comando CLI con resultado (OK/ERROR) y duración.
- Login, logout, cambio de club activo y fallos de autenticación.
- Inicio/fin de transacciones críticas (carga partido, importación Excel, backup/restore).
- Creación/actualización de entidades principales (club, jugador, competencia, partido).
- Excepciones de dominio y errores de infraestructura, clasificados por severidad.

### Criterios de Aceptación

**AC1.** Niveles DEBUG/INFO/WARNING/ERROR en formato JSON estructurado.
**AC2.** `correlation_id` UUIDv4 generado al inicio de cada comando y propagado hasta repositorios.
**AC3.** Secretos (passwords, tokens, hashes sensibles) redactados en todos los niveles de log.
**AC4.** Selección de sink (local-file o Seq) por configuración sin cambiar código de negocio.
**AC5.** En modo Seq, eventos filtrables por `correlation_id`, `command_name` y nivel.

### Testing Mínimo

- *Unitario:* creación y propagación de `execution_context`; validación de campos obligatorios.
- *Unitario:* redacción de secretos en campos conocidos (password, token, session_id).
- *Integración:* flujo CLI completo → verificar que cada evento tiene el mismo `correlation_id`.

## H1-E3 · Entorno de Calidad y Pipeline CI/CD NUEVA

## US-108 — Entorno de Desarrollo y Pipeline CI/CD NUEVA

**Esfuerzo:** S (1–2 días) **Prioridad:** Alta **Dependencias:** —

### Objetivo Funcional

Garantizar que todo nuevo commit sea verificado automáticamente con linting, tests y cobertura, haciendo del pipeline la única fuente de verdad sobre el estado de calidad del proyecto.

### Archivos a Crear

- `.github/workflows/ci.yml` — Pipeline CI en GitHub Actions. Se ejecuta en cada push y PR. Pasos: checkout → setup Python → `pip install` → `flake8` → `pytest -cov` → reporte de cobertura. Falla el PR si algún paso falla.
- `pytest.ini` — Configura pytest: directorio de tests, markers personalizados (`unit`, `integration`), umbral de cobertura con `-cov-fail-under=80`.
- `.flake8` — Reglas de estilo: `max-line-length=120`, exclusión de carpetas generadas. Coherente con el formateador que elija el equipo (black/autopep8).
- `.pre-commit-config.yaml` — Hooks que se ejecutan antes de cada commit local: `flake8` (lint), `black` (formato), `isort` (orden de imports), `check-yaml` (archivos YAML válidos). Se instala una vez con `pre-commit install`.
- `pyproject.toml` — Metadatos del proyecto, versión inicial (`0.1.0`), dependencias de producción y de desarrollo separadas. Punto de entrada del ejecutable: `statspro = "src.infrastructure.ui.cli.main_cli:main"`.
- `Makefile` — Atajos para el equipo: `make test` (pytest), `make lint` (flake8), `make coverage` (pytest+html report), `make install-hooks` (pre-commit install), `make clean` (elimina `.pyc`, `__pycache__`).
- `docs/catalogo-criticidad.md` — Tabla inicial con los módulos del Hito 1 clasificados como críticos/no críticos, su justificación y cobertura objetivo.
- `docs/guias/01-flujo-de-trabajo-git.md` — Guía del flujo de ramas del proyecto: convención de nombres, cuándo hacer PR, cómo hacer code review, y qué verificar antes de mergear.

### Criterios de Aceptación

**AC1.** `make test` ejecuta la suite completa y falla si cobertura <80 % en módulos no críticos.
**AC2.** `make lint` falla el CI ante cualquier infracción de estilo.
**AC3.** El pipeline CI falla el PR ante test fallido, cobertura insuficiente o error de linting.
**AC4.** `pre-commit install` configura hooks locales en un único comando.
**AC5.** `docs/catalogo-criticidad.md` inicializado con al menos los módulos de autenticación y persistencia.

### Testing Mínimo

- *Manual:* push a rama feature → el workflow de CI se ejecuta y reporta correctamente.
- *Manual:* introducir error de lint intencionalmente → CI falla; corregir → CI pasa.
- *Manual:* bajar cobertura por debajo del umbral → CI falla con mensaje explicativo.

## 9.2 Hito 2 — Motor de Ingesta y Análisis

**H2 · Motor de Ingesta y Análisis** v0.2

Automatizar la carga desde Ges Deportivo, producir métricas avanzadas confiables, exponer estadísticas por CLI y asegurar la evolución controlada del esquema de base de datos.

**Épicas:** 3 **Historias:** 5 **Esfuerzo total:** ≈30 días · persona

### H2-E1 · Integración Ges Deportivo

#### US-201 — Parseo de Planillas Excel (Ges Deportivo)

**Esfuerzo:** L (6–10 días) **Prioridad:** Urgente BLOQUEANTE **Dependencias:** US-105, ADR-003

**Objetivo Funcional**

Convertir planillas Excel externas de Ges Deportivo en datos persistibles sin transcripción manual, incluyendo modo *preview* sin efecto secundario y reporte de calidad de la importación.

**Archivos a Crear**

- `src/infrastructure/ingest/excel_parser.py` — Lee el archivo Excel con `pandas.read_excel()`. Mapea las columnas del formato Ges Deportivo a los nombres internos del sistema, convierte tipos (fechas, enteros) y detecta filas malformadas. Retorna un `DataFrame` normalizado o una lista de errores de parsing.
- `src/infrastructure/ingest/ingest_service.py` — Orquesta el proceso de merge: itera filas del DataFrame, decide si crear o vincular jugador (por DNI), prepara los DTOs y llama al caso de uso `CargarPartido`. Maneja el rollback si falla alguna fila.
- `src/application/use_cases/importar_excel.py` — Caso de uso que recibe la ruta del archivo y un flag `preview: bool`. En modo preview solo retorna el reporte de calidad sin persistir; en modo commit ejecuta la importación real.
- `src/application/dtos/ingest_dto.py` — `IngestReportDTO`: contiene listas de `creados`, `actualizados`, `rechazados` (con causa), y totales. Es lo que se muestra al usuario al finalizar la importación.
- `tests/integration/test_excel_import.py` — Tests con el fixture real de Ges Deportivo.
- `tests/fixtures/ges_deportivo_sample.xlsx` — Archivo Excel de ejemplo con el formato real de Ges Deportivo. Incluye casos borde: fila sin DNI, puntos inconsistentes, jugador ya existente.
- `docs/guias/02-formato-ges-deportivo.md` — Documentación del formato esperado del Excel: columnas requeridas/opcionales, tipos de dato, ejemplos de filas válidas e inválidas, y cómo generar el archivo de prueba.

**Criterios de Aceptación**

**AC1.** Validación previa de columnas obligatorias y tipos mínimos esperados antes de cualquier persistencia.

**AC2.** Modo `preview` genera reporte de filas válidas/erróneas sin escribir en la base de datos.
**AC3.** Merge por DNI: crea jugador nuevo si no existe; vincula a partido si existe.
**AC4.** Consistencia entre suma de puntos individuales y marcador de partido (validación cruzada).
**AC5.** Importación atómica por partido: fallo en cualquier fila → rollback de ese partido completo.
**AC6.** Reporte final con cantidades de creados, actualizados, rechazados y causa de rechazo por fila.
**AC7.** Filas sin DNI se rechazan explícitamente y quedan registradas en el reporte de calidad.
**AC8.** Clubes o competencias faltantes se crean de forma controlada y trazable (con log de operación).

### Testing Mínimo

- *Unitario — parser:* mapeo de columnas, tipado correcto y detección de errores de formato.
- *Unitario — ingest-service:* estrategias de merge (nuevo/existente) y rechazo de filas inválidas.
- *Integración:* importación de fixture a DB `:memory:`; verificar persistencia y rollback ante error.
- *Integración:* modo `preview` no produce cambios; comparar estado antes/después.

## H2-E2 · Motor Estadístico Avanzado

## US-202 — Cálculo de Métricas Avanzadas

**Esfuerzo:** M (3–5 días) **Prioridad:** Alta **Dependencias:** US-201

### Objetivo Funcional

Transformar el boxscore crudo en indicadores tácticos accionables mediante funciones puras determinísticas, manteniendo la capa de fórmulas libre de toda dependencia de base de datos.

### Archivos a Crear

- `src/infrastructure/analytics/formulas.py` — Funciones puras que reciben un `DataFrame` y retornan una serie o escalar. Cada función corresponde a una fórmula (eFG %, EFF, PPP, posesiones, %rebotes). Sin acceso a DB, sin efectos secundarios: mismos inputs → mismos outputs siempre.
- `src/application/use_cases/calcular_estadisticas_avanzadas.py` — Obtiene el boxscore del partido via `AnalyticsService`, aplica las fórmulas de `formulas.py` y retorna un `MetricasDTO` con todos los indicadores calculados.
- `src/application/use_cases/generar_tabla_comparativa.py` — Calcula métricas para ambos equipos del partido y retorna un `ComparativaDTO` con una fila por equipo.
- `src/application/dtos/metricas_dto.py` — `MetricasDTO`: campos para cada métrica calculada (eFG %, EFF, PPP, etc.) con sus valores redondeados. `ComparativaDTO`: dos `MetricasDTO`, uno por equipo.
- `tests/unit/test_formulas.py` — Un test por fórmula con valores calculados a mano para

verificar exactitud. Tests de casos borde (0 posesiones, 0 tiros).

- `docs/arquitectura/05-motor-estadistico.md` — Documentación de cada fórmula: nombre, definición matemática, referencia bibliográfica (Basketball Reference, FIBA), y ejemplo numérico paso a paso.

#### Fórmulas Mínimas

- **eFG %** (Effective Field Goal Percentage)
- **EFF** (Efficiency Index)
- **PPP** (Points Per Possession)
- **Posesiones estimadas** (fórmula de posesiones por equipo)
- **Porcentaje de rebotes** (ofensivo y defensivo)

#### Criterios de Aceptación

**AC1.** Cada fórmula documentada con su fuente/referencia técnica y salida determinística.
**AC2.** División por cero y valores `NaN`/`inf` normalizados explícitamente (sin propagación silenciosa).
**AC3.** Filtros disponibles por temporada, competencia y rango de partidos.
**AC4.** Reporte comparativo Equipo vs. Rival disponible por partido.
**AC5.** `formulas.py` no contiene ningún acceso a base de datos ni I/O externo.

#### Testing Mínimo

- *Unitario:* cobertura de cada fórmula con valores conocidos (valores de referencia calculados a mano).
- *Unitario:* casos borde: posesiones = 0, tiros = 0, rebotes = 0.
- *Integración:* use cases con datos semilla y comparación contra valores esperados precalculados.

### US-203 — Integración del Motor Analítico (Pandas Engine)

**Esfuerzo:** M (3–5 días) **Prioridad:** Alta **Dependencias:** US-202

---

#### Objetivo Funcional

Conectar las vistas SQL con los servicios analíticos sin acoplar fórmulas y persistencia, estableciendo un contrato de `AnalyticsService` desacoplado de detalles de infraestructura.

#### Archivos a Crear

- `src/domain/repositories/analytics_service.py` — Interfaz abstracta del servicio analítico. Declara métodos como `get_boxscore_partido(partido_id)` y `get_totales_temporada(club_id, temporada)`. Vive en `domain/repositories/` porque es un port que la aplicación consume.
- `src/infrastructure/analytics/pandas_analytics_service.py` — Implementación concreta. Lee desde las vistas SQL via `pandas.read_sql()`, aplica las fórmulas de `formulas.py` y retorna DataFrames con columnas estandarizadas.

- `src/application/use_cases/calcular_estadisticas_partido.py` — Orquesta la obtención de datos y el cálculo para un partido específico.
- `tests/integration/test_analytics_service.py` — Tests end-to-end desde vistas SQL hasta DTO de métricas, usando la base de datos semilla.

#### Criterios de Aceptación

**AC1.** El servicio lee exclusivamente desde `v_boxscore_completo` y `v_jugador_totales_temporada`.
**AC2.** El contrato `AnalyticsService` en dominio no tiene referencias a infraestructura.
**AC3.** Columnas de salida estandarizadas y documentadas para consumo de UI y reportería.
**AC4.** Errores de lectura o cálculo retornan excepciones de dominio controladas, nunca excepciones de Pandas.

#### Testing Mínimo

- *Unitario:* DataFrames de prueba en memoria con mock del repositorio de lectura.
- *Integración:* flujo completo desde vistas SQL hasta DTO de métricas con datos semilla.
- *Regresión:* columnas esperadas en vistas consumidas (test que falla si se renombra una columna).

### US-205 — Consulta Estadística por CLI NUEVA

**Esfuerzo:** S (1–2 días) **Prioridad:** Media **Dependencias:** US-203, US-106

#### Objetivo Funcional

Exponer las métricas del motor analítico directamente desde la CLI, permitiendo al DT consultar líderes, estadísticas de un partido y comparativas de equipo sin necesidad de la interfaz gráfica.

#### Archivos a Crear

- `src/infrastructure/ui/cli/commands/stats_commands.py` — subcomandos `stats show`, `stats leaders`, `stats compare`.
- `src/infrastructure/ui/cli/formatters/stats_formatter.py` — formateador tabular de métricas avanzadas.
- `tests/integration/test_stats_commands.py`

#### Comandos CLI

- `stats show partido <id>` — boxscore + métricas avanzadas del partido.
- `stats leaders <temporada>[-top N]` — top-N por PTS, REB, AST, EFF.
- `stats compare <partido_id>` — comparativa equipo vs. rival.

#### Criterios de Aceptación

**AC1.** Cada comando produce salida tabular legible con alineación numérica correcta.
**AC2.** Filtros `-temporada` y `-competencia` funcionan en `stats leaders`.
**AC3.** Partido inexistente devuelve mensaje descriptivo, no traceback.

**AC4.** Todos los valores muestran consistencia con los calculados por `formulas.py`.

#### Testing Mínimo

- *Integración:* cada subcomando con datos semilla; verificar columnas y valores en salida.
- *Integración:* partido/temporada inexistente → mensaje de error sin excepción no controlada.

## H2-E3 · Gobernanza de Esquema y Migraciones

### US-204 — Versionado de Esquema y Migraciones

**Esfuerzo:** M (3–5 días) **Prioridad:** Alta **Dependencias:** US-101, ADR-004

#### Objetivo Funcional

Asegurar la evolución controlada del esquema de base de datos entre versiones sin pérdida de datos, con un runner que aplique migraciones en orden y registre cada resultado.

#### Archivos a Crear

- `src/infrastructure/persistence/sql/migrations/001_init.sql` — Primera migración: el schema completo de v0.1. A partir de aquí, el schema no se modifica directamente, solo se agregan nuevos archivos de migración.
- `src/infrastructure/persistence/sql/migrations/002_*.sql` — Migraciones incrementales. Convención de nombre: `NNN_descripcion_corta.sql`. Cada archivo es idempotente (usa `IF NOT EXISTS` o `IF EXISTS` según corresponda).
- `src/infrastructure/persistence/migration_runner.py` — Al arrancar la app, consulta la tabla `schema_version`, detecta qué migraciones no fueron aplicadas, las ejecuta en orden y registra el resultado. Si una falla, hace rollback y bloquea el arranque con un mensaje claro.
- `tests/integration/test_migrations.py` — Prueba el upgrade multi-versión y el rollback.
- `docs/guias/03-como-agregar-una-migracion.md` — Guía paso a paso: cuándo crear una migración, cómo nombrarla, qué debe contener, cómo probarla localmente y cómo verificar que no rompe datos existentes.

#### Criterios de Aceptación

**AC1.** Tabla `schema_version` creada y mantenida automáticamente con número de versión, nombre y timestamp.

**AC2.** Runner ejecuta solo migraciones pendientes (idempotente en migraciones ya aplicadas).

**AC3.** Soporte de rollback controlado para la última migración aplicada (script `down` opcional).

**AC4.** Al arrancar la app, si el schema es incompatible con la versión del código, la ejecución se bloquea con mensaje claro.

**AC5.** Scripts versionados con el mismo estándar de nomenclatura y registrados en el changelog técnico.

#### Testing Mínimo

- *Integración:* upgrade multi-versión sobre base con datos reales; verificar cardinalidad conservada.
- *Integración:* rollback de la última migración; verificar integridad post-rollback.
- *Regresión:* dataset de versión anterior conserva todas las relaciones tras migrar.

## 9.3 Hito 3 — Visualización Pro y Reporting

**H3 · Visualización Pro y Reporting** v0.3

Convertir las estadísticas en tableros, gráficos e informes consumibles por el DT para tomar decisiones tácticas antes, durante y después del partido.

**Épicas:** 3 **Historias:** 3 **Esfuerzo total:** ≈22 días · persona

### H3-E1 · Visualización y Dashboards

#### US-301 — Dashboards e Informes Interactivos

**Esfuerzo:** M (3–5 días) **Prioridad:** Alta **Dependencias:** US-203

**Objetivo Funcional**

Exponer líderes estadísticos y tendencias en CLI/GUI con filtros tácticos, permitiendo al DT identificar rápidamente a los jugadores más determinantes de la temporada.

**Archivos a Crear**

- `src/application/use_cases/obtener_lideres_temporada.py`
- `src/application/use_cases/generar_grafico_rendimiento.py`
- `src/infrastructure/analytics/chart_generator.py` — generación de gráficos (librería según ADR-007).
- `src/infrastructure/reports/cli_leaderboard_reporter.py`
- `tests/unit/test_leaderboard.py`
- `tests/integration/test_chart_generator.py`

**Criterios de Aceptación**

**AC1.** Top-5 por rubro (PTS, REB, AST, EFF) en formato tabular legible con posiciones y valores.

**AC2.** Gráficos de tendencia por jugador/equipo filtrables por temporada y competencia.

**AC3.** Generación de gráfico en menos de 2 segundos para dataset de referencia (3 temporadas, 20 equipos, 1.200 filas de boxscore).

**AC4.** Valores graficados coinciden con los valores calculados por las vistas SQL (consistencia garantizada).

**Testing Mínimo**

- *Unitario:* ordenamiento de líderes y desempate por criterio secundario.
- *Unitario:* transformación de DataFrame a serie temporal para gráficos.
- *Integración:* reporter CLI + chart generator con datos semilla; verificar salida completa.

## H3-E2 · Reportería y Exportación

### US-302 — Exportación Profesional a PDF

**Esfuerzo:** M (3–5 días) **Prioridad:** Alta **Dependencias:** US-301, ADR-005

#### Objetivo Funcional

Emitir un reporte formal de partido o temporada en formato PDF para análisis y difusión interna del cuerpo técnico.

#### Archivos a Crear

- `src/infrastructure/reports/pdf_generator.py` — generación con librería según ADR-005 (ej. ReportLab).
- `src/application/use_cases/exportar_reporte.py`
- `src/application/dtos/reporte_dto.py`
- `tests/integration/test_pdf_export.py`

#### Criterios de Aceptación

**AC1.** Encabezado con nombre de clubes, competencia, fecha y metadata del reporte.
**AC2.** Tablas de boxscore y métricas avanzadas con alineación numérica y cabeceras claras.
**AC3.** Nombre de archivo con convención: `boxscore_YYYY-MM-DD_Local_vs_Visitante.pdf`.
**AC4.** Directorio `exports/` creado automáticamente si no existe.
**AC5.** Errores de I/O (ruta sin permisos, disco lleno) producen mensaje de recuperación, no crash.

#### Testing Mínimo

- *Integración:* generación de PDF válido (no corrupto, abre con lector estándar) desde partido semilla.
- *Integración:* verificar nombre de archivo y ruta de salida esperada.
- *Integración:* ruta sin permisos de escritura → excepción controlada con mensaje útil.

## H3-E3 · Inteligencia de Scouting

### US-303 — Scouting de Rival Pre-Partido

**Esfuerzo:** M (3–5 días) **Prioridad:** Media **Dependencias:** US-203

**Objetivo Funcional**

Entregar al DT un reporte táctico detallado del rival antes del encuentro, basado en el historial de partidos de la temporada.

**Archivos a Crear**

- `src/infrastructure/persistence/sql/views_scouting.sql` — vistas de análisis histórico por rival.
- `src/application/use_cases/generar_scouting_rival.py`
- `src/application/dtos/scouting_dto.py`
- `tests/integration/test_scouting_rival.py`

**Criterios de Aceptación**

**AC1.** Ventana de análisis de últimos $N$ partidos configurable por parámetro.
**AC2.** Detección de patrones: rachas (victorias/derrotas consecutivas), tendencia de tiro y pérdidas recurrentes.
**AC3.** Filtros por competencia y categoría producen resultados consistentes con las vistas SQL.
**AC4.** Reporte exportable (texto/PDF) con conclusiones destacadas y datos fuente citados.

**Testing Mínimo**

- *Unitario:* agregaciones históricas y funciones de detección de rachas.
- *Integración:* vistas de scouting con dataset histórico de ejemplo (mínimo 10 partidos del rival).
- *Integración:* consistencia de filtros por competencia y ventana $N$.

## 9.4 Hito 4 — Interfaz Multiplataforma y Entrega

**H4 · Interfaz Multiplataforma y Entrega** v1.0

Migrar a experiencia visual completa con Flet, cerrar el hardening de seguridad, garantizar continuidad operativa mediante backup/restore y distribuir la aplicación como binario autónomo para las plataformas objetivo.

**Épicas:** 4 **Historias:** 4 **Esfuerzo total:** ≈35 días · persona

**H4-E1 · Interfaz Gráfica Multiplataforma**

**US-401 — Implementación de Interfaz Flet (Desktop/Mobile)**

**Esfuerzo:** L (6–10 días) **Prioridad:** Alta **Dependencias:** US-301, US-302, ADR-002

**Objetivo Funcional**

Ofrecer una interfaz visual completa reutilizando los casos de uso consolidados en hitos previos, operativa en desktop y mobile sin modificar la lógica de negocio.

### Archivos a Crear

- `src/infrastructure/ui/flet/app.py` — entry point y configuración de la app Flet.
- `src/infrastructure/ui/flet/screens/dashboard_screen.py` — pantalla principal.
- `src/infrastructure/ui/flet/screens/game_entry_screen.py` — carga de partidos.
- `src/infrastructure/ui/flet/screens/player_profile_screen.py` — perfil de jugador.
- `src/infrastructure/ui/flet/screens/import_screen.py` — importación de Excel.
- `src/infrastructure/ui/flet/components/chart_component.py` — gráficos embebidos.
- `src/infrastructure/ui/flet/components/stats_table_component.py` — tablas reutilizables.
- `tests/unit/test_flet_validators.py`

### Criterios de Aceptación

**AC1.** Arranque inicial <3 segundos en entorno objetivo (desktop, 4 GB RAM, SSD, Python 3.11+).

**AC2.** Navegación estable entre pantallas principales sin pérdida de estado.

**AC3.** Soporte de tema claro/oscuro y diseño responsive para desktop y mobile.

**AC4.** Formularios con validación visual (mensajes de error en campo, sin modal genérico).

**AC5.** La UI no contiene lógica de negocio; invoca casos de uso via inyección de dependencias.

### Testing Mínimo

- *Unitario:* validadores de formularios y reglas de navegación de estado de pantalla.
- *Manual guiado:* flujo Dashboard → GameEntry → PlayerProfile en desktop y mobile.
- *Manual:* importación de Excel desde pantalla de importación con archivo de prueba.

## H4-E2 · Resiliencia y Backup

## US-402 — Backup y Restauración de Base de Datos

**Esfuerzo:** M (3–5 días) **Prioridad:** Alta **Dependencias:** US-204, ADR-008

---

### Objetivo Funcional

Proteger la continuidad operativa mediante un mecanismo de exportación e importación de la base de datos completa, con verificación automática de integridad post-restauración.

### Archivos a Crear

- `src/application/use_cases/exportar_backup.py`
- `src/application/use_cases/restaurar_backup.py`
- `src/infrastructure/persistence/backup_service.py`
- `tests/integration/test_backup_restore.py`

### Criterios de Aceptación

**AC1.** Exportación completa incluye metadata: versión del schema, fecha/hora y hash de integri-

dad.

**AC2.** Restauración funciona tanto sobre instancia vacía como sobre instancia con datos existentes.

**AC3.** Verificación automática post-restore: conteo de filas en tablas críticas y checksum lógico de datos clave.

**AC4.** Checklist de estabilización ejecutada y documentada (rendimiento, errores críticos, regresión).

### Testing Mínimo

- *Integración:* ciclo completo exportar → restaurar → verificar igualdad de datos clave.
- *Integración:* restauración desde backup de versión anterior con migraciones aplicadas automáticamente.
- *Regresión:* ejecución de casos críticos tras restore (auth, carga partido, exportación PDF).

## H4-E3 · Seguridad de Datos Local

## US-403 — Hardening de Seguridad Local

**Esfuerzo:** M (3–5 días) **Prioridad:** Alta **Dependencias:** US-104, ADR-006

### Objetivo Funcional

Endurecer credenciales, política de sesión y almacenamiento local para que la aplicación cumpla con un nivel de seguridad adecuado al contexto de datos deportivos personales.

### Archivos a Crear

- `src/infrastructure/security/credential_policy.py` — validación de fortaleza y política de contraseña.
- `src/infrastructure/security/compromised_password_store.py` — lista local versionada de contraseñas comprometidas.
- `src/infrastructure/security/db_encryption_adapter.py` — adaptador SQLCipher (opcional).
- `docs/security/password_compromised_update_procedure.md` — procedimiento de actualización mensual.
- `tests/unit/test_credential_policy.py`

### Criterios de Aceptación

**AC1.** Contraseñas mínimo de 12 caracteres con validación de complejidad (mayúscula, número, símbolo).

**AC2.** Lista local de contraseñas comprometidas bloquea efectivamente su uso en registro y cambio.

**AC3.** Procedimiento de actualización mensual documentado en `docs/security/` con pasos re-

producibles.

**AC4.** CI valida versión vigente y checksum de la lista de comprometidas en cada PR.

**AC5.** Hashing robusto (bcrypt o Argon2) con salt único por usuario; migración de hashes legados.

**AC6.** Cifrado de base de datos local configurable (SQLCipher) sin romper la API del `DatabaseManager`.

**AC7.** Política de sesión: expiración configurable, revocación inmediata en logout y limpieza idempotente.

**AC8.** Eventos de seguridad (fallos de login, cambios de contraseña) trazados en log sin exponer datos sensibles.

#### Testing Mínimo

- *Unitario:* política de contraseñas (longitud, complejidad, lista comprometidas).
- *Unitario:* ciclo hash → verificación; migración de hash legado a nuevo algoritmo.
- *Integración:* expiración y revocación de sesión; estado DB tras logout.
- *Integración:* acceso a DB con y sin cifrado según configuración.

## H4-E4 · Empaquetado y Distribución NUEVA

### US-404 — Empaquetado y Distribución Multiplataforma NUEVA

**Esfuerzo:** M (3–5 días) **Prioridad:** Alta **Dependencias:** US-401, US-108

#### Objetivo Funcional

Generar binarios autónomos distribuibles para cada plataforma objetivo (Windows, Linux, macOS, Android) sin requerir instalación de Python ni dependencias externas por parte del usuario final.

#### Archivos a Crear

- `build/build_desktop.py` — Script PyInstaller que genera un ejecutable autónomo para Windows/Linux/macOS. Configura el nombre del ejecutable, el ícono, y excluye módulos innecesarios para reducir tamaño.
- `build/build_android.py` — Script de empaquetado Flet para Android (usa `flet build apk`). Configura el `package_id`, versión y permisos necesarios.
- `build/build_ios.py` — Ídem para iOS (`flet build ipa`). Requiere macOS y Xcode.
- `.github/workflows/release.yml` — Pipeline de release. Se activa con un tag `v*.*.*`. Pasos: tests → build para cada plataforma → upload de artefactos → crear GitHub Release automáticamente con las notas del `CHANGELOG.md`.
- `pyproject.toml` — Actualizado con la versión de release, entrypoints CLI y dependencias de producción separadas de las de desarrollo.
- `CHANGELOG.md` — Historial de versiones en formato *Keep a Changelog*. Ver Sección 13 para

el proceso completo.

- `docs/install-guide.md` — Instrucciones de instalación para cada plataforma: descargar el binario, permisos necesarios, primer uso.

**Criterios de Aceptación**

**AC1.** Binario desktop lanza la aplicación sin Python instalado en el sistema del usuario.
**AC2.** APK de Android instalable en dispositivos Android 9+ sin dependencias adicionales.
**AC3.** El pipeline de release genera artefactos para las 3 plataformas desktop en cada tag de versión.
**AC4.** Versión embebida en la aplicación (About screen/CLI `-version`) coincide con el tag de release.
**AC5.** `CHANGELOG.md` actualizado siguiendo formato *Keep a Changelog* en cada release.

**Testing Mínimo**

- *Manual:* instalar binario desktop en máquina limpia (sin Python); ejecutar flujo completo.
- *Manual:* instalar APK en dispositivo Android real o emulador; verificar arranque y operación básica.
- *CI:* pipeline de release ejecuta tests completos antes de generar artefactos.

## 10 Reglas de Negocio Consolidadas

**Objetivo:** centralizar las reglas obligatorias del dominio para que el desarrollo y testing sean coherentes en todas las historias de usuario.

### Identidad y Seguridad

- Email de usuario único por sistema.
- Contraseña nunca persistida en texto plano ni en logs.
- Sesión requiere usuario autenticado y, para comandos operativos, club activo.

### Jugadores, Clubes y Afiliaciones

- DNI de jugador único cuando está informado.
- Un jugador no puede tener dos vínculos activos superpuestos con el mismo club.
- Historial de afiliación coherente en fechas: `fecha_hasta >= fecha_desde`.

### Competencias, Inscripciones y Listas

- Una inscripción es única por club + competencia + categoría + temporada.
- Cada inscripción tiene una única lista de buena fe asociada.
- Solo jugadores habilitados en lista pueden figurar en carga oficial de partido.

### Partidos y Estadísticas

- Un partido no puede tener el mismo club como local y visitante.
- Toda carga de partido y boxscore es atómica (todo o nada).

- Estadísticas de tiro: convertidos ≤ lanzados; todos los valores son no negativos.
- Puntos de jugador = T1C + T2C×2 + T3C×3 (coherencia verificada).
- Minutos por jugador no pueden exceder el máximo reglamentario definido por competencia.

### Analítica y Reporting

- Fórmulas avanzadas deben manejar división por cero y no devolver `NaN/inf`.
- Reportes y dashboards se construyen sobre vistas SQL normalizadas y versionadas.
- Toda exportación (PDF/backup) debe ser trazable en logs con timestamp y resultado.

## 11 Requisitos No Funcionales (NFR)

| ID | Requisito | Medición / Umbral | Severidad |
|---|---|---|---|
| **NFR-1** | Portabilidad | Ejecución nativa en Windows 10+, Android 9+, iOS 14+. | Bloqueante |
| **NFR-2** | Rendimiento Ingesta | Procesamiento de Excel con Pandas <5 seg. | Alta |
| **NFR-3** | Fiabilidad de Datos | Integridad referencial en SQLite (FKs activas). | Bloqueante |
| **NFR-4** | Usabilidad | Carga de partido completo en <3 clics desde selección de archivo. | Media |
| **NFR-5** | Arranque | Tiempo de inicio de la GUI <3 seg. en entorno objetivo. | Alta |
| **NFR-6** | Offline | 100 % de funcionalidades críticas sin conexión a internet. | Bloqueante |
| **NFR-7** | Cobertura de Tests | ≥80 % no críticos; ≥95 % en módulos críticos (ver catálogo). | Alta |

## 12 Definición de "Hecho" (Definition of Done)

Una historia de usuario se considera **Hecha** cuando:

1. El código está integrado en la rama principal sin conflictos de merge.
2. Pasa tests unitarios e integración con cobertura ≥ 80 % en componentes no críticos.
3. Componentes críticos alcanzan cobertura ≥ 95 % (autenticación, persistencia transaccional, rollback, fórmulas).
4. La clasificación crítico/no crítico está definida y versionada en `docs/catalogo-criticidad.md`.
5. Se respeta la arquitectura de capas (sin imports cruzados entre `domain` e `infrastructure`).
6. Si hay cambios SQL, la vista/tabla está versionada y cubierta por test de integración con datos semilla.
7. Si la US persiste múltiples tablas, existe evidencia de transacción atómica y rollback probado.
8. El ADR correspondiente fue aprobado y documentado.
9. Si la US incluye UI, fue validada manualmente en al menos dos plataformas (desktop + mobile).

10. Métodos públicos relevantes incluyen docstring y trazabilidad mínima en logs de operación.
11. El pipeline CI pasa en verde (lint + tests + cobertura).

### 12.1 Catálogo Técnico de Criticidad

El catálogo de criticidad es un artefacto vivo del backlog de arquitectura donde se lista cada módulo con: nivel de criticidad, justificación, owner técnico y cobertura objetivo.

**Ubicación:** `docs/catalogo-criticidad.md` — tabla mínima: `Módulo | Criticidad | Justificación | Owner | Cobertura objetivo | Evidencia`.

**Criterios de clasificación (crítico si cumple al menos uno):**

- Controla acceso, autenticación o autorización.
- Persiste datos en múltiples tablas con transacciones.
- Impacta la integridad histórica del dato deportivo.
- Implementa cálculos estadísticos oficiales usados en decisiones tácticas.

**Gobernanza:** owner primario: Arquitecto Técnico; co-owner: QA Lead. Revisión obligatoria en kickoff de hito y en cada planificación de sprint. El catálogo se inicializa en **US-108** y se actualiza en cada planificación.

## 13 Proceso de Liberación de Versiones

Esta sección documenta el flujo completo para publicar una nueva versión del proyecto en GitHub, desde la preparación de la rama hasta la creación del Release con artefactos.

### 13.1 Versionado Semántico

El proyecto sigue **Semantic Versioning 2.0** con el formato `vMAJOR.MINOR.PATCH`:

- **MAJOR** — cambios incompatibles con versiones anteriores (ej. rediseño del schema que requiere migración manual).
- **MINOR** — nuevas funcionalidades compatibles con versiones anteriores. **Cada hito completado incrementa el MINOR** (H1 → v0.1.0, H2 → v0.2.0, H4 → v1.0.0).
- **PATCH** — corrección de bugs sin nuevas funcionalidades (ej. v0.1.1, v0.1.2).

**Versiones pre-1.0**

Mientras `MAJOR = 0`, la API no se considera estable. Cualquier hito puede introducir cambios en el schema o en los contratos internos. La versión `1.0.0` se alcanza al completar el Hito 4.

## 13.2 Estrategia de Ramas

| Rama | Propósito |
|---|---|
| `main` | Solo contiene commits de merge desde ramas `release/`. Cada commit en `main` tiene un tag de versión. Nunca se trabaja directamente aquí. |
| `develop` | Rama de integración. Las `feature/` se mergean aquí. Es la rama "siguiente versión en construcción". |
| `feature/nombre` | Una rama por Historia de Usuario o tarea. Se crea desde `develop` y se mergea a `develop` via PR. Ejemplo: `feature/us-101-sqlite-schema`. |
| `release/vX.Y.Z` | Se crea desde `develop` cuando el hito está feature-complete. Solo acepta commits de corrección de bugs y actualización de versión. |
| `hotfix/descripcion` | Para bugs críticos en producción. Se crea desde `main` y se mergea tanto a `main` como a `develop`. |

## 13.3 Proceso de Release Paso a Paso

**1. Preparar la rama de release**

- Crear la rama: `git checkout -b release/v0.1.0 develop`
- Actualizar la versión en `pyproject.toml`: campo `version = "0.1.0"`.
- Actualizar `CHANGELOG.md`: mover los cambios de `[Unreleased]` a `[0.1.0] - YYYY-MM-DD`.
- Ejecutar la suite completa de tests: `make test` (debe pasar en verde).
- Corregir únicamente bugs críticos encontrados. No agregar features nuevas.

**2. Mergear a `main` y crear el tag**

- Mergear: `git checkout main` → `git merge -no-ff release/v0.1.0`
- Buscar comando git para agregar tag
- El mensaje del tag describe el contenido del hito, no solo el número.
- Subir main y el tag: `git push origin main -tags`

**3. Mergear de vuelta a `develop`**

- `git checkout develop` → `git merge -no-ff release/v0.1.0`
- Esto asegura que los fixes del release lleguen a la rama de desarrollo.
- Eliminar la rama de release: `git branch -d release/v0.1.0`

**4. Crear el GitHub Release**

Con la GitHub CLI (`gh`):

- `gh release create v0.1.0 -title "v0.1.0 - Núcleo de Datos e Interfaz CLI-notes-file CHANGELOG.md`
- Para adjuntar binarios generados por el pipeline: `gh release upload v0.1.0 dist/statspro-v0.1.0-windows.`
- Alternativamente, el workflow `.github/workflows/release.yml` (US-404) hace esto automáticamente cuando se detecta el tag.

**5. Verificar el release**

- Confirmar en GitHub que el release aparece con el tag correcto, las notas del changelog y los artefactos adjuntos.

- Instalar el binario en una máquina limpia y ejecutar el smoke-test de instalación.

### 13.4 Formato del `CHANGELOG.md`

El proyecto usa el estándar *Keep a Changelog* (`keepachangelog.com`). Estructura básica:

- `[Unreleased]` — sección al tope del archivo donde se registran los cambios en desarrollo. Se convierte en la sección del próximo release al liberar una versión.
- `[vX.Y.Z] - YYYY-MM-DD` — una sección por release con subsecciones:
  - `Added` — nuevas funcionalidades.
  - `Changed` — cambios en funcionalidades existentes.
  - `Fixed` — corrección de bugs.
  - `Removed` — funcionalidades eliminadas.
  - `Security` — correcciones de seguridad.

**Buena práctica**

Actualizar `[Unreleased]` en cada PR mergeado, no solo al momento del release. Así el changelog refleja el progreso real y el release no requiere reconstruir la historia de cambios de memoria.

### 13.5 Hotfix: Corregir un Bug en Producción

1. Crear desde `main`: `git checkout -b hotfix/descripcion-breve main`
2. Aplicar la corrección mínima necesaria. Un test de regresión que falla sin el fix.
3. Actualizar el `PATCH` en `pyproject.toml` (ej. `0.1.0` → `0.1.1`) y el `CHANGELOG.md`.
4. Mergear a `main`: `git checkout main` → `git merge -no-ff hotfix/descripcion-breve`
5. Crear tag: `git tag -a v0.1.1 -m "Hotfix: descripción del bug corregido"`
6. Mergear también a `develop` para no perder el fix.
7. Crear GitHub Release con la nota del bug corregido.

## 14 Hoja de Ruta y Futuras Expansiones

Las siguientes fases de expansión post-v1.0 están diseñadas para transformar StatsPro Basketball en una plataforma líder de inteligencia deportiva formativa.

**Estado de los Hitos Futuros**

Los hitos 5–9 son **visión de producto**, no compromisos de entrega. Sus funcionalidades serán refinadas y priorizadas en función del feedback real de usuarios al cierre de v1.0.

**Hito 5 · Ecosistema Conectado y Colaborativo** v1.5

**Objetivo:** romper el aislamiento local para permitir colaboración entre cuerpos técnicos y difusión institucional de resultados.

- **Sincronización Cloud Híbrida.** Backend FastAPI + PostgreSQL para respaldo en la nube y uso multidispositivo (PC en oficina, tablet en cancha). La arquitectura local-first se mantiene: la app funciona 100 % offline y sincroniza cuando hay conexión.

- **Módulo de Exportación para Redes Sociales.** Generación automática de "Match Cards" con las figuras del partido (líder en puntos, rebotes, eficiencia) en formato optimizado para Instagram/X del club.
- **Base de Datos Compartida de Scouting.** Repositorio comunitario donde los DTs pueden compartir datos históricos de rivales (bajo consentimiento explícito), creando un mapa de scouting regional colaborativo.

### ★ Hito 6 · Inteligencia Táctica con IA v2.0

**Objetivo:** pasar del análisis descriptivo (*¿qué pasó?*) al análisis prescriptivo (*¿qué debería pasar?*) mediante modelos de aprendizaje automático entrenados sobre datos reales del equipo.

- **Motor de Recomendaciones de Quinteto.** Modelos de *machine learning* (scikit-learn/XGBoost) que sugieren el cinco inicial óptimo basándose en el *matchup* contra el rival, el rendimiento reciente y la fatiga acumulada. El pipeline incluye: recolección de features por partido → entrenamiento offline → inferencia en tiempo real al seleccionar rival.
- **Detección Automática de Patrones.** Análisis de ventanas deslizantes sobre el historial del equipo para identificar:
  - Rachas de tiro frío/caliente por zona del campo.
  - Degradación defensiva en cuartos finales (por fatiga).
  - Pérdidas recurrentes en situaciones de presión (últimos 2 minutos).

  Algoritmos: z-score para anomalías, ARIMA para series temporales de rendimiento.
- **Análisis Prescriptivo Explicable.** Los modelos incluyen un módulo de explicabilidad (SHAP values) que permite al DT entender *por qué* se recomienda una acción, no solo *qué* acción tomar. Esto es crítico para la adopción en contextos donde la confianza del entrenador en el sistema es fundamental.
- **Integración de Video Scouting.** Vinculación de eventos estadísticos con clips de video (integración con archivos locales o YouTube) para sesiones de video-análisis rápidas basadas en datos.
- **Marco de Experimentación A/B.** Permite al DT comparar dos estrategias de alineación a lo largo de la temporada y medir su impacto en métricas objetivas (PPP, eFG %, diferencial de puntos).

### ★ Hito 7 · Live Stats y Cancha Digital v3.0

**Objetivo:** capturar datos en el momento en que ocurren, eliminando la dependencia exclusiva de planillas externas y habilitando decisiones tácticas en tiempo real.

- **Módulo de Captura en Vivo.** Interfaz táctil optimizada para tablet que permite a un asistente registrar eventos (tiro intento/convertido por zona, pérdida de balón, falta, rebote)

con latencia <200ms por acción. Diseño de un solo toque para no interrumpir el ritmo del partido.

- **Live Dashboard para el Banco.** Pantalla secundaria (proyectada en tablet del DT) con métricas avanzadas actualizadas en tiempo real: PPP por posesión, eFG % acumulado, parciales por cuarto y +/- por jugador en cancha. Diseñado para decisiones en tiempos muertos.
- **Mapa de Tiro Interactivo.** Visualización de la cancha con zonas de tiro coloreadas por eficiencia (eFG %) del equipo y del rival, actualizado tiro a tiro durante el partido.
- **Control de Carga y Fatiga.** Registro de minutos por jugador contra umbrales configurables por categoría y posición. Alertas automáticas cuando un jugador joven supera el límite de carga recomendado en torneos intensos.

### ★ Hito 8 · Plataforma Federativa y Competición Multi-Liga v4.0

**Objetivo:** escalar de la herramienta de un club a una plataforma de gestión estadística oficial para federaciones y ligas regionales.

- **Integración con APIs Federativas.** Conexión con los endpoints oficiales de federaciones nacionales (CABB, FIBA) para importar fixtures, plantillas habilitadas y enviar estadísticas oficiales de cada partido automáticamente, eliminando la doble carga de datos.
- **Vista Multi-Club para Oficiales de Liga.** Panel de administración que permite a un director de competencia ver estadísticas agregadas de todos los clubes de una liga, identificar anomalías (datos faltantes, inconsistencias) y aprobar planillas digitalmente.
- **Reportería Regulatoria Automatizada.** Generación de los informes estadísticos requeridos por la federación en los formatos oficiales (PDF/Excel), eliminando el trabajo manual de recopilación post-temporada.
- **Benchmarking Anónimo Inter-Club.** Métricas de rendimiento comparativas entre clubes de la misma categoría (anonimizadas), permitiendo a cada DT saber en qué percentil de la liga se encuentra su equipo en cada indicador.
- **Gestión de Árbitros y Planilleros.** Módulo de asignación de árbitros por partido con estadísticas de su desempeño (tiempo de juego efectivo, faltas cobradas) para la federación.

### ★ Hito 9 · Multi-Deporte y Ecosistema de Formación v5.0

**Objetivo:** extender el motor estadístico más allá del básquet y convertir la plataforma en un ecosistema integral de desarrollo deportivo formativo.

- **Motor Estadístico Multi-Deporte.** Arquitectura modular que permite definir nuevos deportes mediante configuración (métricas, posiciones, reglas) sin cambiar el núcleo. Extensiones iniciales: voleibol, fútbol sala y handball, con sus fórmulas específicas (ej. rotaciones en voleibol, efectividad en zonas en fútbol sala).

- **Seguimiento de Desarrollo a Largo Plazo.** Trazabilidad del crecimiento de un jugador a lo largo de múltiples temporadas y categorías (infantil → cadete → junior → mayor), con gráficos de evolución de métricas clave y proyecciones de desarrollo.
- **Modelado de Riesgo de Lesión.** Integración del volumen de carga acumulada (minutos, intensidad estimada) con datos de desempeño para detectar señales tempranas de sobreentrenamiento en jugadores en formación, con alertas para el cuerpo médico.
- **Módulo de Formación para Entrenadores.** Vinculación de situaciones estadísticas identificadas (ej. deficiencia en rebotes ofensivos) con recursos pedagógicos categorizados por nivel de certificación FIBA. El DT recibe sugerencias de ejercicios de entrenamiento basadas en las debilidades estadísticas de su equipo.
- **Portal de Seguimiento para Familias.** Interfaz simplificada (solo lectura, opt-in por familia) donde los padres de jugadores en categorías formativas pueden ver las estadísticas públicas de su hijo y su evolución a lo largo de la temporada, fomentando el vínculo club-familia.